You already completed this chapter.

# 3.1 Function Types and Signatures

Not completed

## Function Type Expressions 📝 (Extra documentation)

> “ No video for this lesson, but plenty of code examples and explanations! ”

**Real-World Analogy:** A function type is like a **job description** - it specifies:

What inputs are needed (parameters)

What output is produced (return type)

The job title (function name/type)

### Why This Matters 💡

**Reusable types** - define function shapes once, use everywhere

**Type safety** - ensure functions match expected signatures

**Better documentation** - function types are self-documenting

**Callback types** - perfect for event handlers and async operations

---

## Basic Function Type Syntax 🔤

```
1  // Function type expression syntax: (param: Type) => ReturnType
2
3  // Simple function type
4  let greet: (name: string) => string;
5
6  greet = function(name: string): string {
7      return `Hello, ${name}!`;
8  };
9
```

```
13
14 // Multiple parameters
15 let add: (a: number, b: number) => number;
16
17 add = (a, b) => a + b;  // TypeScript infers parameter types
18
19 console.log(add(5, 10));  // 15
20
21 // No parameters
22 let getMessage: () => string;
23
24 getMessage = () => "Hello, World!";
25
26 console.log(getMessage());  // "Hello, World!"
```

### Function Types with Objects

```
1  // Function that returns an object
2  let createUser: (name: string, age: number) => { name: string; age: number };
3
4  createUser = (name, age) => {
5      return { name, age };
6  };
7
8  const user = createUser("Alice", 28);
9  console.log(user);  // { name: "Alice", age: 28 }
10
11 // Function that takes an object
12 let displayUser: (user: { name: string; age: number }) => void;
13
14 displayUser = (user) => {
15     console.log(`${user.name} is ${user.age} years old`);
16 };
17
18 displayUser(user);  // "Alice is 28 years old"
```

### Creating Reusable Function Types

```typescript
// Define function type alias
type GreetFunction = (name: string) => string;

// Use the type alias
const greet: GreetFunction = (name) => `Hello, ${name}!`;
const welcome: GreetFunction = (name) => `Welcome, ${name}!`;

// Function that takes a callback
function processName(name: string, formatter: GreetFunction): string {
    return formatter(name);
}

console.log(processName("Alice", greet));     // "Hello, Alice!"
console.log(processName("Bob", welcome));     // "Welcome, Bob!"
```

### Complex Function Type Aliases

```typescript
// Calculator operation type
type Operation = (a: number, b: number) => number;

const add: Operation = (a, b) => a + b;
const subtract: Operation = (a, b) => a - b;
const multiply: Operation = (a, b) => a * b;
const divide: Operation = (a, b) => a / b;

function calculate(a: number, b: number, op: Operation): number {
    return op(a, b);
}

console.log(calculate(10, 5, add));       // 15
console.log(calculate(10, 5, subtract));  // 5
console.log(calculate(10, 5, multiply));  // 50
console.log(calculate(10, 5, divide));    // 2
```

## Basic Function Overloads

What are overloads?

In the code example below, the three lines at the top (the declarations without bodies) are overload signatures. They tell TypeScript: "There is a function named `format` that can be called with a string, or a number, or a boolean."

Overloads shape the function's public API, ie what callers are allowed to pass.

Why an implementation signature?

The actual function body must be one implementation that handles all permitted input types. That is the line:

```
function format(value: string | number | boolean): string { ... }
```

The implementation uses a union type (string | number | boolean) so it can accept any of the overload inputs. TypeScript requires the implementation signature to be compatible with every overload.

What callers see and why that matters

From outside, TypeScript uses the overload signatures to provide editor help and type checking. For example, calling `format("abc")` is valid and its type is `string`.

Even though numbers and booleans are handled, the return type for every overload is `string`. That means `format(123)` is typed as `string`.

```
// Define the overload signatures
function format(value: string): string;
function format(value: number): string;
function format(value: boolean): string;

// Implementation signature (must be compatible with all overloads)
function format(value: string | number | boolean): string {
    if (typeof value === "string") {
        return `"${value}"`;
    } else if (typeof value === "number") {
        return value.toFixed(2);
    } else {
        return value ? "true" : "false";
    }
```

```
console.log(format(42));        // "42.00"
console.log(format(true));      // "true"

// TypeScript knows which overload you're using
const str = format("test");   // Type: string
const num = format(123);      // Type: string
```

### Overloads with Different Parameter Counts

```
// Overload signatures
function createElement(tag: string): HTMLElement;
function createElement(tag: string, content: string): HTMLElement;
function createElement(tag: string, children: HTMLElement[]): HTMLElement;

// Implementation
function createElement(
    tag: string,
    contentOrChildren?: string | HTMLElement[]
): HTMLElement {
    const element = document.createElement(tag);

    if (typeof contentOrChildren === "string") {
        element.textContent = contentOrChildren;
    } else if (Array.isArray(contentOrChildren)) {
        contentOrChildren.forEach(child => element.appendChild(child));
    }

    return element;
}

// Usage
const div1 = createElement("div");
const div2 = createElement("div", "Hello");
const div3 = createElement("div", [
    createElement("span", "Item 1"),
    createElement("span", "Item 2")
]);
```

```
function getUser(id: number): { id: number; name: string };
function getUser(email: string): { email: string; name: string };

// Implementation
function getUser(
    idOrEmail: number | string
): { id: number; name: string } | { email: string; name: string } {
    if (typeof idOrEmail === "number") {
        return { id: idOrEmail, name: "User " + idOrEmail };
    } else {
        return { email: idOrEmail, name: "User with email" };
    }
}

const userById = getUser(123);
console.log(userById.id);  // ✅ OK - TypeScript knows this has id

const userByEmail = getUser("alice@example.com");
console.log(userByEmail.email);  // ✅ OK - TypeScript knows this has email
```

## Best Practices 🏆

### Use Type Aliases for Repeated Function Types

```
// ✅ Good - reusable type
type Validator = (value: string) => boolean;

const emailValidator: Validator = (value) => value.includes("@");
const lengthValidator: Validator = (value) => value.length >= 8;

// ❌ Bad - repeated type
const emailValidator2: (value: string) => boolean = (value) => value.includes("
const lengthValidator2: (value: string) => boolean = (value) => value.length >=
```

### Prefer Union Types Over Overloads When Possible

```
}

// ❌ Unnecessarily complex
function process2(value: string): void;
function process2(value: number): void;
function process2(value: string | number): void {
    console.log(value);
}
```

### Use Overloads for Different Behaviors

```
// ✅ Good - overloads make sense here
function createElement(tag: "div"): HTMLDivElement;
function createElement(tag: "span"): HTMLSpanElement;
function createElement(tag: string): HTMLElement;
function createElement(tag: string): HTMLElement {
    return document.createElement(tag);
}

const div = createElement("div");    // Type: HTMLDivElement
const span = createElement("span");  // Type: HTMLSpanElement
```

Not completed ⊗